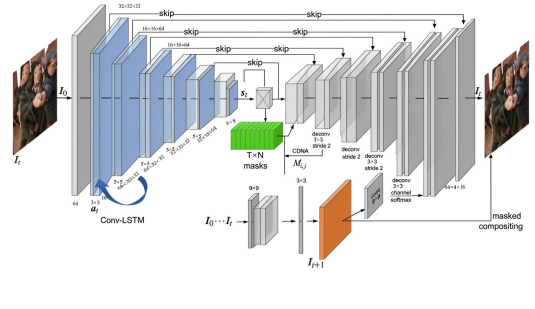
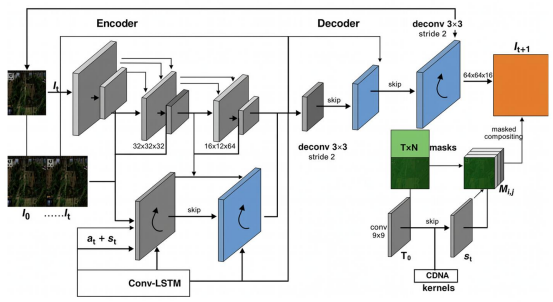
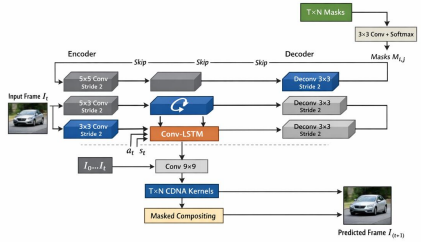
SciForma-9B	Wan-2.7-Image	GPT-Image-1.5

The figure illustrates a general Spatio-Temporal Neural Architecture (SNA) model designed for video prediction or frame generation, based on the mathematical formulation referenced in the paper's Equation (general_model). The architecture is structured as an encoder-decoder network with skip connections and incorporates temporal modeling via a Conv-LSTM module, along with mask-based compositing for generating the next frame. [1] Global Layout and Structure: The diagram is horizontally oriented, depicting a left-to-right data flow. On the far left, the current input frame I_t enters the encoder. The encoder progressively downsamples the spatial resolution through convolutional layers, reducing dimensions from 64x64 to 8x8. At the bottleneck, the feature maps are fed into a Conv-LSTM cell, which processes temporal context from previous frames I_0...I_t. The decoder then upsamples the features back to the original resolution using deconvolutional layers. Parallel to this, a separate branch generates T×N masks, which are used in conjunction with learned CDNA kernels to composite the predicted frame I_{t+1}. Skip connections link corresponding encoder and decoder layers to preserve spatial details. [2] Visual Modules and Attributes: - The encoder consists of multiple 3D blocks representing convolutional layers. The first layer is a 5×5 convolution with stride 2, followed by several 3×3 convolutions with stride 2, reducing spatial dimensions while increasing channel depth (from 64 to 32 to 16). These blocks are colored gray or blue; blue blocks contain a curved arrow symbol indicating they are part of the Conv-LSTM module. - The Conv-LSTM module is explicitly labeled at the bottom left and receives the encoded features along with temporal inputs a_t and s_t (likely the hidden and cell states from prior time steps). - The decoder mirrors the encoder with deconvolutional layers (labeled 'deconv 3×3 stride 2'), gradually restoring spatial resolution. The final deconvolutional layer outputs 64×64×16 features. - A separate green block labeled 'T×N masks' is generated alongside the decoder output. This block undergoes a 3×3 convolution with channel softmax to produce normalized masks M_{i,j}. - Below the main path, a 9×9 convolutional layer takes the sequence I_0...I_t and produces T×N CDNA kernels. These kernels are combined with the masked compositing operation to generate the final output frame I_{t+1}, shown as an orange rectangle. - The output frame I_{t+1} is produced via 'masked compositing', which combines the CDNA kernels with the masks M_{i,j}. [3] Connections and Arrows: - The input I_t flows through the encoder, with each convolutional layer connected sequentially. Skip connections (labeled 'skip') link encoder layers to corresponding decoder layers (e.g., 32×32×32 to 32×32×32, 16×16×64 to 16×16×64), preserving spatial information. - The bottleneck features are fed into the Conv-LSTM, which also receives a_t and s_t from previous time steps. The output of the Conv-LSTM feeds into the decoder. - The decoder layers are connected via deconvolutional operations, increasing spatial resolution step-by-step. - From the final decoder layer, two paths diverge: one leads to the T×N masks (green block), and the other connects to the CDNA kernel generator (gray block with 'conv 9×9'). - The CDNA kernels are combined with the masks M_{i,j} via masked compositing to produce I_{t+1}. - The entire process is framed as a recurrent structure, where past frames I_0...I_t influence the prediction of I_{t+1} through both the Conv-LSTM and CDNA kernel computation.







The figure presents an overview of the Vision Transformer (ViT) architecture, illustrating its end-to-end processing pipeline for image classification. The global layout is divided into two main sections: the left side shows the full ViT model structure, while the right side provides a detailed breakdown of the Transformer Encoder block, separated by a vertical dashed line. On the left, the process begins at the bottom with an input image, which is divided into a grid of non-overlapping patches. These patches are shown as small image tiles, and are then passed through a 'Linear Projection of Flattened Patches' module, represented as a light pink rectangular box. This step converts each patch into a fixed-dimensional vector. Above this, a sequence of ten embedded tokens (numbered 0* through 9) is displayed, where token 0* is marked as an extra learnable [class] embedding, distinct from the other patch embeddings. These tokens are labeled 'Patch + Position Embedding', indicating that positional information has been added to the flattened patch vectors. All these embeddings are fed into a large gray rectangular block labeled 'Transformer Encoder'. The output of the Transformer Encoder is connected to an 'MLP Head' (a light orange rectangle), which processes the final representation, particularly the class token (token 0*), to produce a classification result. The output is shown as a rounded orange box listing possible classes such as 'Bird', 'Ball', 'Car', etc., indicating the model's prediction. On the right, the internal structure of the 'Transformer Encoder' is expanded. It is depicted as a gray box containing a stack of layers repeated L times, indicated by 'L x' at the top. The input to this block is 'Embedded Patches' (light pink rectangle). The first layer is a 'Norm' (yellow rectangle), followed by a 'Multi-Head Attention' module (green rectangle). The output of this attention layer is added to the input via a residual connection (indicated by a circular '+' symbol and a feedback arrow). The result is then passed through another 'Norm' (yellow), followed by an 'MLP' (light blue rectangle). Another residual connection adds the MLP output back to the pre-MLP state, completing one encoder layer. This entire stack is repeated L times, and the final output is passed upward to the next stage in the ViT pipeline. All connections are represented by solid black arrows, indicating the forward flow of data. The color coding helps distinguish components: pink for input/patch-related modules, yellow for normalization layers, green for attention, blue for MLP, and gray for the main encoder block. The figure effectively abstracts the ViT method as a sequence of patchification, embedding, position encoding, and transformer-based feature learning, culminating in classification via an MLP head.